

설계대로 구현하고, AI를 통제하다

의료·자동차 보험 시스템

분산프로그래밍 기말 발표

무엇을 만들었나 — 한눈에



의료·자동차보험의 가입부터 청구·지급까지. 액터는 셋 — 고객 · 보험사 직원 · 시스템.
클래스·유스케이스 다이어그램은 직접 설계하고, 구현은 AI로 진행했습니다.

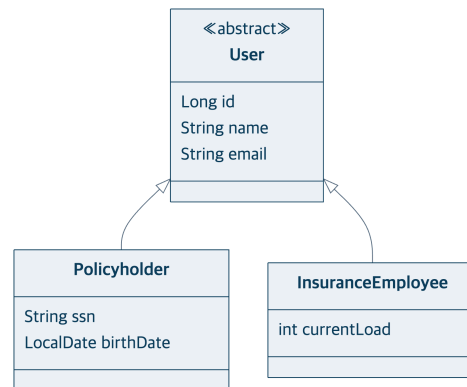
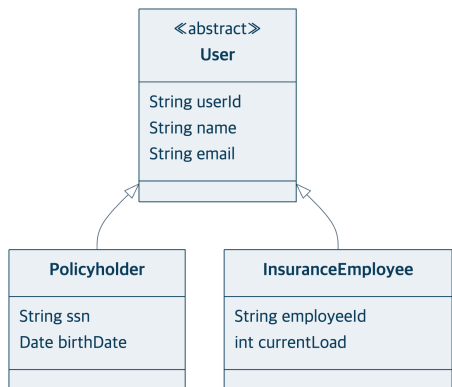
설계 = 구현 ① — 사용자 · 상품 계층

내가 설계 (EA)

코드 역설계 (구현)

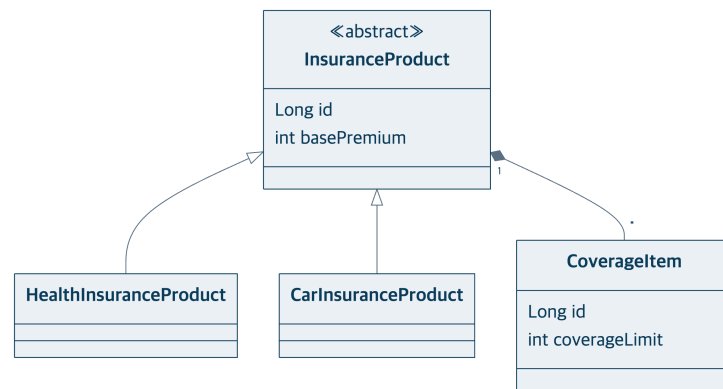
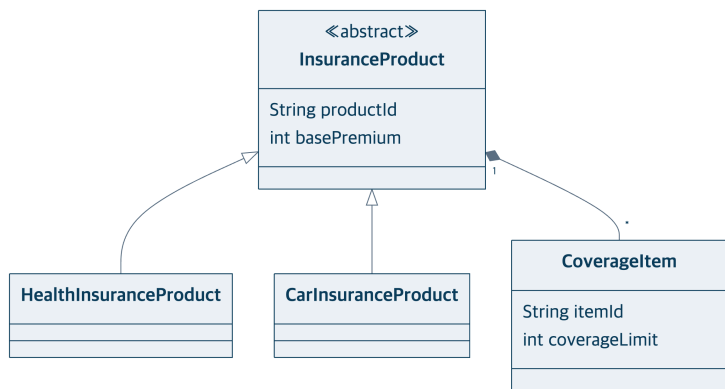
사용자

✓ 일치



상품

✓ 일치



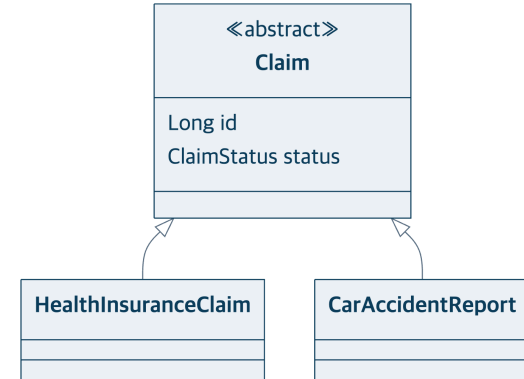
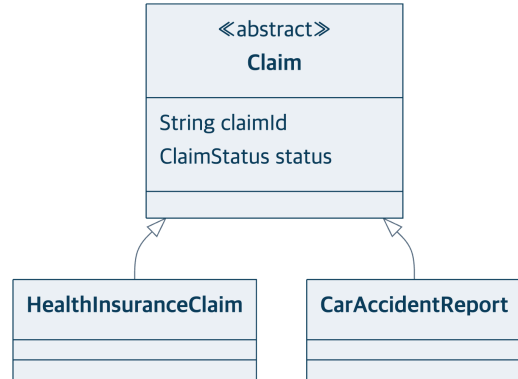
클래스·상속·관계가 동일합니다. 노란 칸의 차이는 식별자(userId→id)·날짜 타입뿐입니다.

설계 = 구현 ② — 청구 · 심사 계층

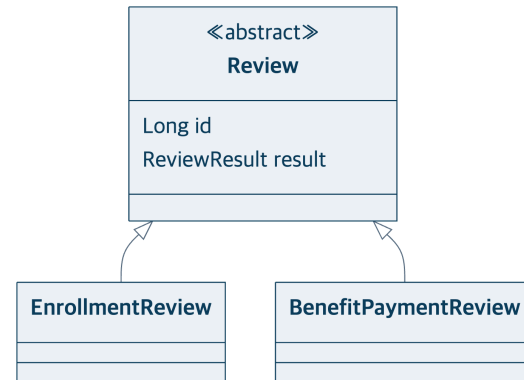
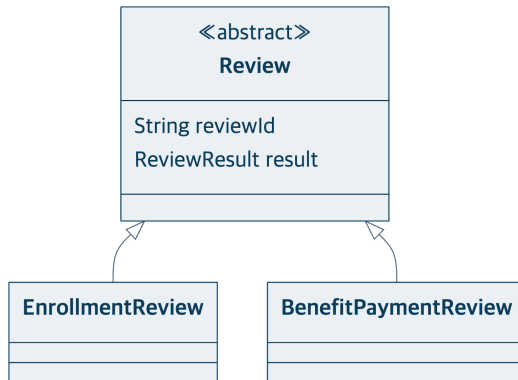
내가 설계 (EA)

코드 역설계 (구현)

청구
✓ 일치



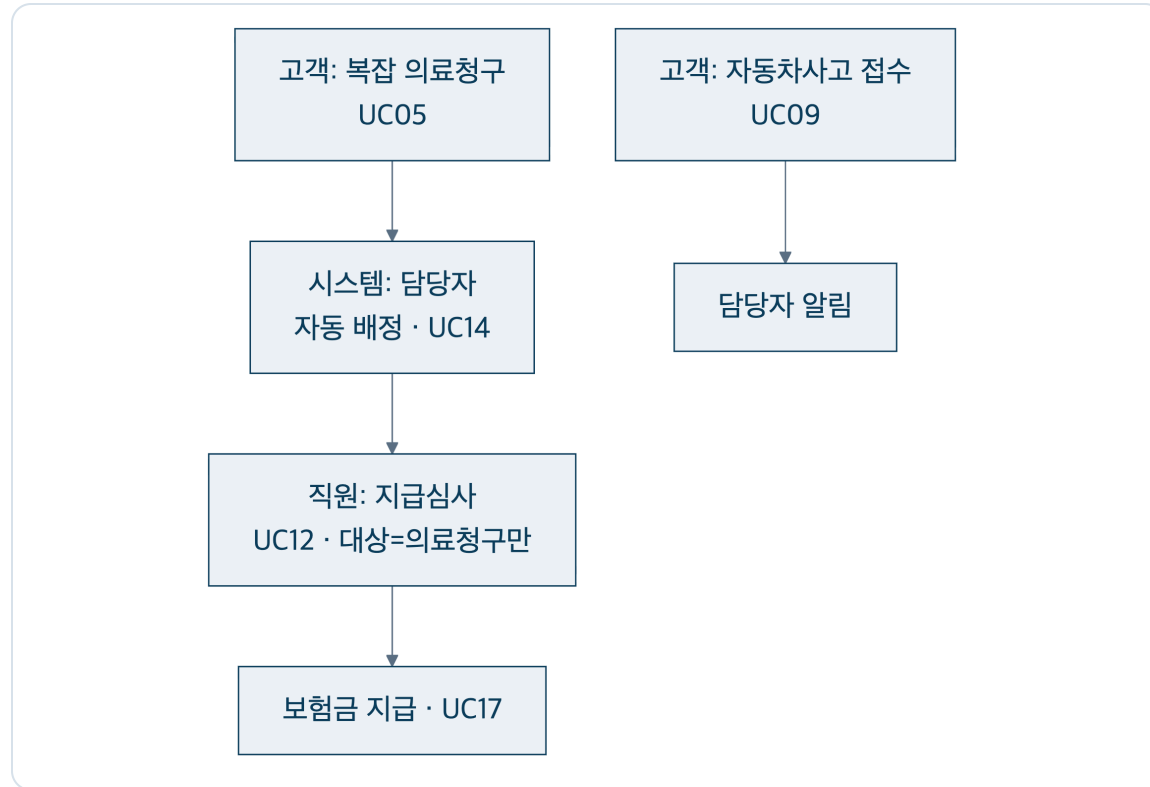
심사
✓ 일치



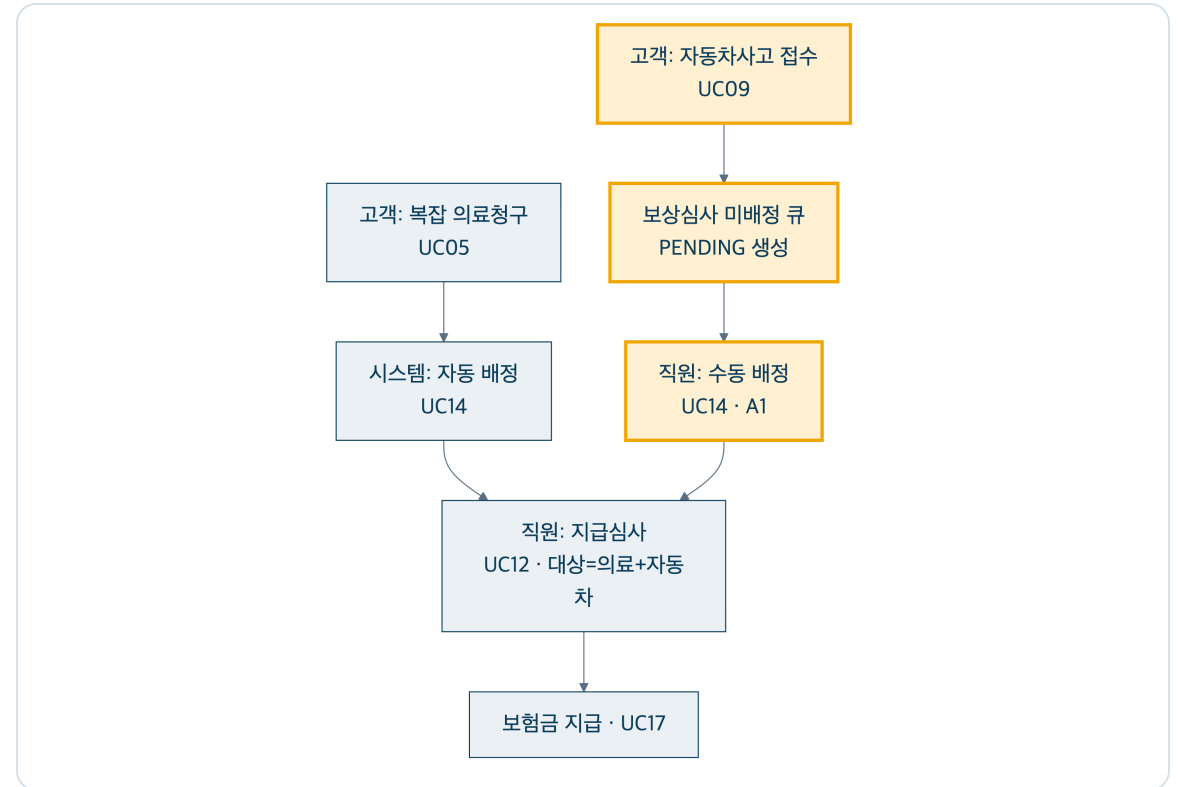
계층을 하나씩 맞춰봐도 — 설계 = 구현.

유스케이스도 명세대로 — 보상 흐름

설계안 (최초)



구현 (데모와 일치)



자동차사고가 보상심사 큐로 진입하고(UC09), 배정이 자동→수동으로 분기되며(UC14), 지급심사 대상이 의료+자동차로 확대(UC12)되었습니다.

데이터는 어떻게 흐르는가 — 한눈에



- 같은 정보가 계층 경계마다 **정해진 한 곳에서만** 형태를 바꿉니다: 폼 → JSON → 요청 DTO → 도메인 객체 → DB, 그리고 역순으로 응답 DTO → 화면.
- **핵심 예시**: 직원이 입력한 **할증률** 하나가 도메인 규칙을 거쳐 **계약의 월 보험료**로 기록됩니다. 화면 표시가 아니라 상태 변화입니다.
- 덕분에 **도메인 모델**은 웹·DB 어느 쪽에도 오염되지 않습니다 (응답 DTO가 엔티티를 격리).

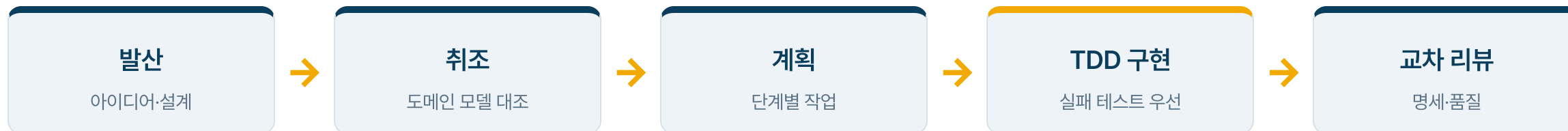
AI를 어떻게 썼는가

사람(설계자)

유스케이스·클래스 다이어그램 · 단계적 진화 전략 ·
되돌리기 어려운 결정(ADR) · 리뷰

AI

다이어그램을 입력으로 받은 구현 · 테스트 작성 · 리팩토링



진화도 한 번에 가지 않고 TUI → DB → 웹 → Spring 4단계로 통제했습니다.

통제 장치: 다이어그램(설계) · 용어집 · ADR · TDD. AI는 구현을 가속했고, 방향은 사람이 정했습니다.

AI를 쓰며 생긴 문제와 해결

① 성급한 추상화

DB 도입 전부터 계층을 과하게 분리

→ 실익 없어 되돌리고, 진화 단계에 맞춰 재도입

② 유스케이스와 불일치

자동차 보상심사를 자동 배정으로 구현

→ 명세 대조(취조)로 발견, 수동 배정으로 정정

③ 계층 패턴 이탈

일부 컨트롤러가 저장소를 직접 호출

→ 전 도메인 패턴 점검으로 식별·관리

④ 용어·도메인 오염

동의어 혼용, 도메인에 출력 코드 삽입 경향

→ 용어집·ADR로 사전 차단 (도메인 출력 0)

AI는 빠르지만 방향은 못 잡습니다. 방향은 사람이 만든 게이트가 잡습니다.

결론

- 직접 설계한 **다이어그램이 데모(구현)와 일치**합니다. 벗어난 지점은 ADR로 통제했습니다.
- 데이터는 **명확한 계층 경계에서만** 형태를 바꾸며 도메인은 오염되지 않습니다.
- **AI는 구현을 가속**했고, **설계·결정·검증은 사람이** 통제했습니다.

감사합니다. 질문 받겠습니다.